Kubernetes Probe 入門
3種類のヘルスチェック完全理解

Readiness / Liveness / Startup Probe

2026-03 · Kubernetes Foundation

Probeが必要な理由 — コンテナ運用の3つのリスク

- 起動中のコンテナにリクエストが流れる　DB接続が完了していないのにトラフィックが到達→エラーが発生
- プロセスが死んでいても気づかない　ハングアップ・メモリリークで応答不能になっても自動復旧しない
- 起動に時間がかかるアプリが誤判定される　遅い起動をヘルスチェック失敗と見なして再起動ループに陥る

SECTION 01

3種類のProbeを理解する

Readiness Probe — 「準備できているか」をチェック

- 役割：Podがリクエストを受け付けられる状態かを確認する
- 失敗時の動作：Serviceのエンドポイントから一時的に除外される　Podはそのまま動き続ける（再起動はしない）
- 典型的なユースケース：アプリ起動済みだがDB接続がまだ完了していない
- 回復すれば自動でエンドポイントに復帰する

Liveness Probe — 「生きているか」をチェック

- 役割：コンテナのプロセスが正常に動作しているかを確認する
- 失敗時の動作：コンテナを強制再起動する　再起動により自動復旧を実現
- 典型的なユースケース：無限ループ・デッドロック・メモリリークによる応答不能
- 定期的に繰り返しチェックし、障害を継続的に検知する

Startup Probe — 「起動完了を待つ」

- 役割：コンテナの起動完了を確認するための専用プローブ
- 動作の仕組み：Startup Probe が成功するまで Liveness / Readiness は実行されない
  起動中に誤って失敗判定されるのを防ぐ
- 典型的なユースケース：起動に時間がかかるが、完了時間が読めないアプリ
  Java アプリ・大規模データ初期化など
- 成功後は Liveness / Readiness に引き継がれ通常の監視が始まる

AWS（ECS）との比較 — おなじみの概念で理解

Kubernetes	AWS / ECS	役割
Readiness Probe	ターゲットグループのヘルスチェック（ALB）	リクエストを受け付ける準備ができたか
Liveness Probe	コンテナの healthCheck 定義	コンテナが生きているかをチェック
Startup Probe	明示的な対応機能なし（起動時間設定で代用）	起動の遅いアプリに猶予を与える

YAML設定例 — Slack Metrics API の実際の設定

```yaml
livenessProbe:          # コンテナが生きているかどうかを確認
  httpGet:
    path: /api/health
    port: 8080
  initialDelaySeconds: 15  # 起動後15秒待ってからチェック開始
  periodSeconds: 10
  failureThreshold: 3

readinessProbe:         # リクエストを受け付けるかどうかを確認
  httpGet:
    path: /api/health
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 10
  failureThreshold: 3
```

Startup Probe vs initialDelaySeconds — 使い分けのポイント

INITIALDELAYSECONDS

- 起動時間が固定・予測可能な場合に有効
- 設定した秒数が経過するまで固定で待機
- 起動が早くても残り時間は無駄に待つ

STARTUP PROBE（推奨）

- 起動時間がブレる場合に有効
- 成功した瞬間に次のProbeへ即移行
- 最大待機時間も柔軟に設定できる

まとめ — 本番環境でのベストプラクティス

- **Readiness Probe** は必ず設定する　未準備のPodへのトラフィックを防ぎ、500エラーを最小化
- **Liveness Probe** は必ず設定する　ハングアップやクラッシュからの自動復旧を保証
- 起動時間が読めないアプリには **Startup Probe** を前段に置く　誤判定による再起動ループを防ぎ、安全に起動完了を待てる
- 3つのProbeはマニフェストで一元管理できる — これがKubernetesの強み